

2026년도 전국기능경기대회 채점기준

1. 채점상의 유의사항	직 종 명	클라우드컴퓨팅
※ 다음 사항을 유의하여 채점하시오. 1) AWS 지역은 각 Module에서 명시된 Region을 사용합니다. 2) 웹페이지 접근은 크롬이나 파이어폭스를 이용합니다. 3) 웹페이지에서 언어에 따라 문구가 다르게 보일 수 있습니다. 4) Shell에서의 명령어의 출력은 버전에 따라 조금 다를 수 있습니다. 5) 문제지와 채점지에 있는 ◁는 변수입니다. 해당 부분을 변경해 입력합니다. 6) 채점은 문항 순서대로 진행해야 합니다. 7) 삭제된 채점 자료는 되돌릴 수 없으므로 유의하여 진행하며, 이의 신청까지 완료 이후 선수가 생성한 클라우드 리소스를 삭제합니다. 8) 부분 점수가 있는 문항은 채점 항목에 부분 점수가 적혀져 있습니다. 9) 부분 점수가 따로 없는 문항은 모두 맞아야 점수로 인정됩니다. 10) 리소스의 정보를 읽어오는 채점항목은 기본적으로 스크립트 결과를 통해 채점을 진행하며, 만약 선수가 이의가 있다면 명령어를 직접 입력하여 확인해볼 수 있습니다. 11) [] 기호는 채점에 영향을 주지 않습니다. 12) 명령어 입력 Box 안의 명령줄은 한 줄 명령어입니다. 별도의 지시가 없으면 수정 없이 박스 안의 전체 내용을 복사하고 셸에 붙여 넣어 명령을 실행합니다. 13) 채점 시 명령어 입력은 CloudShell을 이용할 수 있습니다. 14) 채점 값 출력 시 값의 앞 또는 뒤로 (큰)따옴표가 출력되거나 줄바꿈되어 수 있으나, 해당 현상은 채점 시 무시해도 좋습니다. 15) 채점지에서 페이지 간 줄바꿈되어 일부 잘린 항목이 있을 수 있음에 유의합니다. 16) 별도 명시가 없는 경우, 빨간색으로 표시된 부분은 정확히 일치하는지 확인하고, 파란색으로 표시된 부분은 무시할 수 있습니다.		

2. 채점기준표

1) 주요항목별 배점			직 종 명			클라우드컴퓨팅		
과제 번호	일련 번호	주요항목	배점	채점방법		채점시기		비고
				독립	합의	경기 진행중	경기 종료후	
제 2과제	1	NoSQL	7.5	○			○	
	2	CDN Function	7.5	○			○	
	3	EKS Scaling	7.5	○			○	
	4	Container Logging	7.5	○			○	
합 계			30					

2) 채점방법 및 기준

(경기종료 후 채점)

과제 번호	일련 번호	주요항목	일련 번호	세부항목(채점방법)	배점
제2과제	1	NoSQL	1	Reservation Table 구성	1.5
			2	GSI + Audit Table 구성	1.0
			3	Lambda + Streams Trigger 구성	1.0
			4	EC2 Server 구성	1.0
			5	Conditional Write Test	1.5
			6	Streams 후처리 + Read API Test	1.5
	2	CDN Function	1	S3 Bucket Configuration	1.0
			2	KeyValueStore + CF Functions 구성	1.0
			3	CloudFront Distribution 구성	1.0
			4	A/B Testing – Cookie 강제 시 동작	1.5
			5	A/B Testing – 무작위 할당 & 쿠키 보존	1.5
			6	A/B Testing – KVS 동적 반영 테스트	1.5
	3	EKS Scaling	1	SQS Queue 구성	0.35
			2	EKS Cluster + NodeGroup 구성	0.75
			3	Application Deployment	1
			4	KEDA + ScaledObject	1.2
			5	Karpenter NodePool/NodeClass	1.2
			6	Scale-out Test	1.5
			7	Scale-In Test	1.5
	4	Container Logging	1	EKS Cluster / NodeGroup Configuration	1.0
			2	App/Grafana ALB&TG Configuration	1.0
			3	Kubernetes Workload Configuration	1.0
			4	App API Test	1.5
			5	Log Pipeline + LogQL Test	1.5
			6	Grafana Datasource health + Dashboard	1.5

순번	채점 항목	
공통	공통	0) 채점 유의사항을 정독합니다. 유의사항 16번을 확인 후 진행합니다. 1) 지정된 리전에서 CloudShell을 엽니다. 2) <code>rm -rf ~/.aws</code> 를 진행합니다..
1	사전준비	제 1모듈은 싱가포르 리전(ap-southeast-1)의 CloudShell에서 채점합니다.
	1-1-A (명령어 입력)	<code>aws dynamodb describe-table --table-name bigbae-nosql-reservation-table --region ap-southeast-1 jq -r '.Table.TableName, (.Table.KeySchema[] .KeyType + " " + .AttributeName), (.Table.AttributeDefinitions[] .AttributeName + " " + .AttributeType), .Table.StreamSpecification.StreamViewType, .Table.BillingModeSummary.BillingMode'</code> <code>aws dynamodb describe-continuous-backups --table-name bigbae-nosql-reservation-table --region ap-southeast-1 jq -r '.ContinuousBackupsDescription.PointInTimeRecoveryDescription.PointInTimeRecoveryStatus'</code>
	1-1-A <u>해당 내용 포함</u>	bigbae-nosql-reservation-table HASH train_id RANGE seat_id seat_id S train_id S NEW_AND_OLD_IMAGES PAY_PER_REQUEST ENABLED
	1-2-A (명령어 입력)	<code>aws dynamodb describe-table --table-name bigbae-nosql-reservation-table --region ap-southeast-1 jq -r '.Table.GlobalSecondaryIndexes[] .IndexName, (.KeySchema[] .KeyType + " " + .AttributeName), .Projection.ProjectionType'</code> <code>aws dynamodb describe-table --table-name bigbae-nosql-audit-table --region ap-southeast-1 jq -r '.Table.TableName, (.Table.KeySchema[] .KeyType + " " + .AttributeName)'</code>
	1-2-A <u>정확히 일치</u>	gsi-user-reservations HASH user_id RANGE reserved_at ALL bigbae-nosql-audit-table HASH event_id
	1-3-A (명령어 입력)	<code>aws lambda get-function --function-name bigbae-nosql-reservation-audit --region ap-southeast-1 jq -r '.Configuration.FunctionName, .Configuration.Runtime, (.Configuration.Timeout tostring)'</code> <code>aws lambda list-event-source-mappings --function-name bigbae-nosql-reservation-audit --region ap-southeast-1 jq -r '.EventSourceMappings[] (.EventSourceArn split("/")[1]), .State'</code>

	1-3-A <u>정확히 일치</u>	bigbae-nosql-reservation-audit python3.13 30 bigbae-nosql-reservation-table Enabled
	1-4-A (명령어 입력)	<pre>EC2_IP=\$(aws ec2 describe-instances --region ap-southeast-1 --filters "Name= tag:Name,Values= bigbae-nosql-app-ec2" "Name= instance-state-name,Values= running" --query "Reservations[].Instances[].PublicIpAddress" --output text) echo "EC2 IP" \${EC2_IP} curl -s --max-time 10 -o /dev/null -w "healthcheck %{http_code}\ n" "http://\${EC2_IP}:8080/healthcheck"</pre>
	1-4-A <u>부분 일치</u>	<pre>EC2 IP 13.250.1.43 healthcheck 200</pre>
	1-5-A (명령어 입력)	<pre>I=\$(aws ec2 describe-instances --region ap-southeast-1 --filters Name= tag:Name,Values= bigbae-nosql-app-ec2 Name= instance-state-name,Values= running --query Reservations[].Instances[].PublicIpAddress --output text) T=train-\$(date +%s) S=A1 U=user1 V=user2 R(){ curl -s -w " %{http_code}" -X POST http://\$I:8080/\$1 -H Content-Type:application/json -d "{ \"train_id\ ": \"\$T\", \"seat_id\ ": \"\$S\", \"user_id\ ": \"\$2\" }"; echo; } R reserve \$U; R reserve \$V; R cancel \$V; R cancel \$U</pre>
	1-5-A <u>정확히 일치</u>	<pre>{"seat_id": "A1", "status": "reserved", "version": 1} 200 {"error": "already reserved"} 409 {"error": "not owner"} 409 {"seat_id": "A1", "status": "cancelled"} 200</pre>
	1-6-A (명령어 입력)	<pre>I=\$(aws ec2 describe-instances --region ap-southeast-1 --filters Name= tag:Name,Values= bigbae-nosql-app-ec2 Name= instance-state-name,Values= running --query Reservations[].Instances[].PublicIpAddress --output text) T=train-\$(date +%s) S=B1 U=usr1 P(){ curl -s -X POST http://\$I:8080/\$1 -H Content-Type:application/json -d "{ \"train_id\ ": \"\$T\", \"seat_id\ ": \"\$S\", \"user_id\ ": \"\$U\" }" >/dev/null; } A(){ aws dynamodb scan --table-name bigbae-nosql-audit-table --region ap-southeast-1 jq "[.Items[]]select(.train_id.S = \"\$T\" and .seat_id.S = \"\$S\") length"; } P reserve curl -s http://\$I:8080/my-bookings/\$U jq "[.]select(.train_id = \"\$T\" and .seat_id = \"\$S\") length" curl -s http://\$I:8080/seats/\$T jq "[.]select(.seat_id = \"\$S\")[0].status,[0].user_id = \"\$U\"]" sleep 30;A P cancel curl -s http://\$I:8080/my-bookings/\$U jq "[.]select(.train_id = \"\$T\" and .seat_id = \"\$S\")[0].status,[0].user_id = \"\$U\"]" sleep 30;A</pre>

	1-6-A <u>정확히 일치</u>	1 ["reserved", true] 1 0 2
2	2-0	제 2모듈은 버지니아 북부(us-east-1)리전의 CloudShell에서 채점합니다.
	2-1-A (명령어 입력)	ACCOUNT_ID=\$(aws sts get-caller-identity --query Account --output text) BUCKET="skillsphone-landing-ab-\${ACCOUNT_ID}" echo \${BUCKET} aws s3api list-buckets --region us-east-1 jq -r '[.Buckets[].Name]' grep "skillsphone-landing-ab-" aws s3api list-objects-v2 --bucket \${BUCKET} --region us-east-1 jq -r '[.Contents[].Key] "\ (any(.[]; . == "version-a/index.html")) \ (any(.[]; . == "version-b/index.html"))"' aws s3api get-public-access-block --bucket \${BUCKET} --region us-east-1 jq -r '.PublicAccessBlockConfiguration (all(.[]; . == true) tostring)' aws s3api get-bucket-policy --bucket \${BUCKET} --region us-east-1 jq -r '.Policy fromjson .Statement[0] (((.Principal.Service // "-") == "cloudfront.amazonaws.com") tostring), (((.Condition.StringEquals."AWS:SourceArn" // "-") startswith("arn:aws:cloudfront::")) tostring)'
	2-1-A <u>부분 일치</u>	skillsphone-landing-ab-<ACCOUNT_ID> skillsphone-landing-ab-<ACCOUNT_ID> true true true true true <u>* 빨간색 부분과 파란색 부분 출력값이 같다면 득점</u>
	2-2-A (명령어 입력)	KVS=\$(aws cloudfront list-key-value-stores --region us-east-1 jq -r '.KeyValueStoreList.Items[] select(.Name=="skillsphone-cdn-ab-config").ARN') aws cloudfront-keyvaluestore list-keys --kvs-arn "\$KVS" --region us-east-1 jq -r '.Items sort_by(.Key) .[] "KVS_KV \ (.Key) \ (.Value)"' aws cloudfront describe-function --name skillsphone-cdn-ab-req-fn --stage LIVE --region us-east-1 jq -r --arg k "\$KVS" '.FunctionSummary "ReqFn \ (.Name) \ (.Status) \ (.FunctionConfig.Runtime) \ ((.FunctionConfig.KeyValueStoreAssociations.Items // []) any(.KeyValueStoreARN == \$k))"' aws cloudfront describe-function --name skillsphone-cdn-ab-res-fn --stage LIVE --region us-east-1 jq -r '.FunctionSummary "ResFn \ (.Name) \ (.Status) \ (.FunctionConfig.Runtime)'"
	2-2-A <u>정확히 일치</u>	KVS_KV version_a /version-a/index.html KVS_KV version_b /version-b/index.html KVS_KV weight 0.3

		ReqFn skillsphone-cdn-ab-req-fn DEPLOYED cloudfront-js-2.0 true ResFn skillsphone-cdn-ab-res-fn DEPLOYED cloudfront-js-2.0
2-3-A (명령어 입력)		CP=\$(aws cloudfront list-cache-policies --region us-east-1 jq -r '.CachePolicyList.Items[] select(.CachePolicy.CachePolicyConfig.Name=="sk illsphone-cdn-ab-cache-policy") .CachePolicy.Id') DID=\$(aws cloudfront list-distributions --region us-east-1 jq -r '.DistributionList.Items[] select(.Comment=="skillsphone-cdn-ab-distributi on") .Id') aws cloudfront get-cache-policy --id "\$CP" --region us-east-1 jq -r '.CachePolicy.CachePolicyConfig "\ (.Name) \ (.ParametersInCacheKeyAndForwardedToOrigin.CookiesConfig.CookieBe havior) \ ((.ParametersInCacheKeyAndForwardedToOrigin.CookiesConfig.Cookies.I tems // []) sort join(","))', "\ (.MinTTL) \ (.DefaultTTL) \ (.MaxTTL)" aws cloudfront get-distribution-config --id "\$DID" --region us-east-1 jq -r --arg c "\$CP" '.DistributionConfig.DefaultCacheBehavior as \$b .DistributionConfig.Origins.Items[0] as \$o "ViewerProtocol \ (\$b.ViewerProtocolPolicy)", "\ ((((\$o.OriginAccessControlId // "")) length) > 0) \ (\$b.CachePolicyId == \$c)", "\ ((((\$b.FunctionAssociations.Items // []) select(.EventType=="viewer-request") .FunctionARN) // "-") sub(".*function/"; "")) \ ((((\$b.FunctionAssociations.Items // []) select(.EventType=="viewer-response") .FunctionARN) // "-") sub(".*function/"; ""))"'
2-3-A <u>정확히 일치</u>		skillsphone-cdn-ab-cache-policy whitelist x-sp-ab 0 300 3600 ViewerProtocol redirect-to-https true true skillsphone-cdn-ab-req-fn skillsphone-cdn-ab-res-fn
2-4-A (명령어 입력)		D=\$(aws cloudfront list-distributions --region us-east-1 jq -r '.DistributionList.Items[] select(.Comment=="skillsphone-cdn-ab-distributi on") .DomainName'); RH=\$(curl -s -i --max-time 10 "http://\$D/"); B="" S=""; for v in a b; do R=\$(curl -si -m 10 -b x-sp-ab=\$v "https://\$D/?_RANDOM"); grep -q "version_\$v" <<<"\$R" && B+=" true" B+=" false"; grep -qi "^set-cookie:" <<<"\$R" && S+=" false" S+=" true"; done; echo "cookie_a_b_body\$B"; echo "cookie_a_b_no_setcookie\$S"; echo "\$(echo "\$RH" head -1 grep -qE '30[12]' && echo true echo false) \$(echo "\$RH" grep -i '^location:' grep -q 'https://' && echo true echo false)"
2-4-A <u>정확히 일치</u>		cookie_a_b_body true true cookie_a_b_no_setcookie true true true true
2-5-A (명령어 입력)		D=\$(aws cloudfront list-distributions --region us-east-1 jq -r '.DistributionList.Items[] select(.Comment=="skillsphone-cdn-ab-distributi on") .DomainName') R1=\$(curl -s -i --max-time 10 "https://\$D/?_=\$(date +%s%N)") V=\$(echo "\$R1" grep -i '^set-cookie:' sed -n

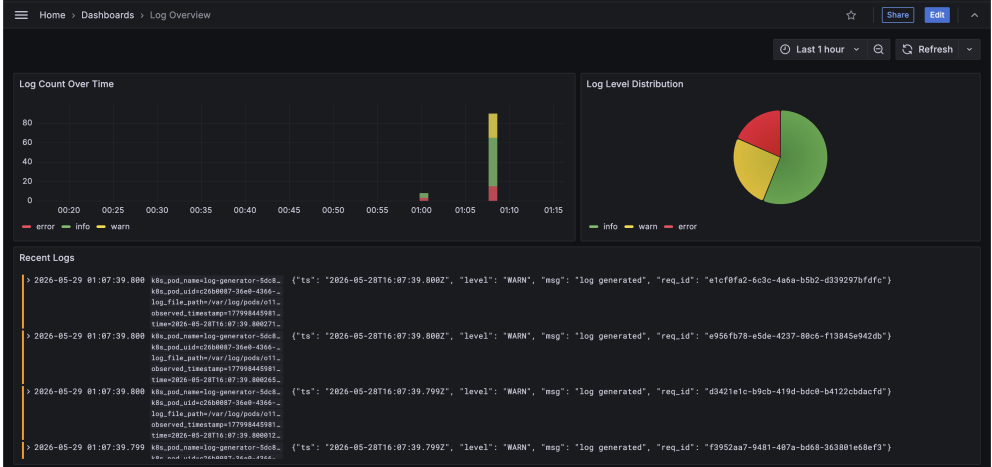
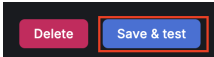
	<pre>'s .x-sp-ab=\ ([ab]\).*\ 1 p') SC=\$(echo "\$R1" grep -i '^ set-cookie:') echo "first_visit_assigned \$V" echo "first_visit_body_match \$(echo "\$R1" grep -q "version_\$V" && echo true echo false)" echo "first_visit_setcookie_maxage \$(echo "\$SC" grep -q 'Max-Age=86400' && echo true echo false)" echo "first_visit_setcookie_path \$(echo "\$SC" grep -q 'Path=/' && echo true echo false)" R2=\$(curl -s -i --max-time 10 -H "Cookie: x-sp-ab=\$V" "https://\$D/?_=\$(date +%s%N)") echo "second_visit_body_match \$(echo "\$R2" grep -q "version_\$V" && echo true echo false)" echo "second_visit_no_setcookie \$([\$(echo "\$R2" grep -ic '^ set-cookie:') -eq 0] && echo true echo false)"</pre>
2-5-A <u>부분 일치</u>	<pre>first_visit_assigned a 또는 b로 출력될 경우 득점 first_visit_body_match true first_visit_setcookie_maxage true first_visit_setcookie_path true second_visit_body_match true second_visit_no_setcookie true</pre>
2-6-A (명령어 입력)	<pre>KVS=\$(aws cloudfront list-key-value-stores jq -r '.KeyValueStoreList.Items[] select(.Name=="skillsphone-cdn-ab-config").ARN') ORIG=\$(aws cloudfront-keyvaluestore get-key --kvs-arn "\$KVS" --key weight jq -r .Value) ETAG=\$(aws cloudfront-keyvaluestore describe-key-value-store --kvs-arn "\$KVS" jq -r .ETag) FETAG=\$(aws cloudfront describe-function --name skillsphone-cdn-ab-req-fn --stage LIVE --query ETag --output text) echo '{"version":"1.0","context":{"eventType":"viewer-request"},"viewer":{"ip":"1.2. 3.4"},"request":{"method":"GET","uri":"/","querystring":{},"headers":{},"cookie s":{}}}' > /tmp/e.json for w in 1.0 0.0; do ETAG=\$(aws cloudfront-keyvaluestore put-key --kvs-arn "\$KVS" --if-match "\$ETAG" --key weight --value "\$w" jq -r .ETag) ["\$w" = "1.0"] && EXP="/version-b/index.html" EXP="/version-a/index.html" TRY=0; while [\$TRY -lt 60]; do URI=\$(aws cloudfront test-function --name skillsphone-cdn-ab-req-fn --if-match "\$FETAG" --stage LIVE --event-object fileb:///tmp/e.json jq -r '.TestResult.FunctionOutput fromjson .request.uri') ["\$URI" = "\$EXP"] && break { sleep 1; TRY=\$((TRY+1)); } done echo "weight_\${w%.*}_uri \$URI"</pre>

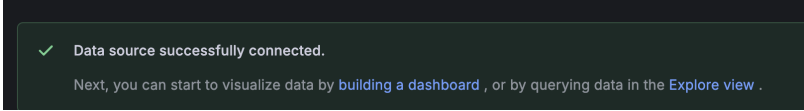
		done aws cloudfront-keyvaluestore put-key --kvs-arn "\$KVS" --if-match "\$ETAG" --key weight --value "\$ORIG" > /dev/null echo "weight_restored \$(aws cloudfront-keyvaluestore get-key --kvs-arn "\$KVS" --key weight jq -r .Value)"
	2-6-A <u>정확히 일치</u>	weight_1_uri /version-b/index.html weight_0_uri /version-a/index.html weight_restored 0.3
3	3-0 사전준비	* 실행 중 에러가 발생할 경우 채점하지 않습니다. source kubectl-connect skm-eks-cluster
	3-1-A (명령어 입력)	aws sqs get-queue-url --queue-name skm-order-queue --region ap-northeast-2 jq .QueueUrl
	3-1-A <u>정확히 일치</u>	"https://sqs.ap-northeast-2.amazonaws.com/123456787890/skm-order-queue"
	3-2-A (명령어 입력)	aws eks describe-cluster --name skm-eks-cluster --query 'cluster.[name, version, status]' --output text --region ap-northeast-2 aws eks describe-nodegroup --cluster-name skm-eks-cluster --nodegroup-name skm-cluster-addon-ng --query 'nodegroup.[instanceTypes[0], scalingConfig.minSize, scalingConfig.desiredSize, scalingConfig.maxSize]' --output text --region ap-northeast-2 aws ec2 describe-instances --filters "Name= tag:Name,Values= skm-cluster-addon-ng-node" "Name= instance-state-name,Values= running" --query "Reservations[].Instances[].Tags[?Key= 'Name'].Value [0]" --output text --region ap-northeast-2
	3-2-A <u>정확히 일치</u>	skm-eks-cluster 1.35 ACTIVE t3.medium 1 1 1 skm-cluster-addon-ng-node
	3-3-A (명령어 입력)	kubectl get pod -n skillsmkt -l app= order-processor -o jsonpath='{.items[0].spec.nodeName}' xargs -l {} kubectl get node {} -L karpenter.sh/nodepool --no-headers kubectl get deploy order-processor -n skillsmkt -o jsonpath='{.spec.template.spec.containers[0].ports[0].containerPort}{.spec.template.spec.containers[0].resources.requests.cpu}{.spec.template.spec.containers[0].resources.requests.memory}' sort kubectl get deploy order-processor -n skillsmkt -o jsonpath='{range .spec.template.spec.containers[0].env[*]}{.name}= {value}' sort
	3-3-A <u>정확히 일치</u>	ip-192-168-133-173.ap-northeast-2.compute.internal Ready <none> 57m v1.35.5-eks-3385e9b skm-app-nodepool 1 8080 500m 512Mi AWS_REGION= ap-northeast-2 PROCESSING_TIME= 3 SQS_QUEUE_URL= https://sqs.ap-northeast-2.amazonaws.com/123456787890/skm-order-queue

3-4-A (명령어 입력)	kubectl get pod -n keda -l app.kubernetes.io/name=keda-operator -o name kubectl get scaledobject order-scaler -n skillsmkt -o jsonpath='{spec.minReplicaCount} {spec.maxReplicaCount} {spec.triggers[0].type} {spec.triggers[0].metadata.queueLength}{\ n"}
3-4-A <u>정확히 일치</u>	pod/keda-operator-55db59458d-tp9bj 1 5 aws-sqs-queue 5
3-5-A (명령어 입력)	kubectl get pod -n kube-system -l app.kubernetes.io/name=karpenter -o name kubectl get nodepool skm-app-nodepool -o jsonpath='{spec.disruption.consolidationPolicy} {spec.disruption.consolidateAfter}{\ n"}' kubectl get nodepool skm-app-nodepool -o json jq -r '.spec.template.spec.requirements[] select(.key == "node.kubernetes.io/instance-type") .values sort join(",")' kubectl get nodepool skm-app-nodepool -o json jq '.spec.template.spec.taints length' kubectl get ec2nodeclass skm-app-nodeclass -o name
3-5-A <u>정확히 일치</u>	pod/karpenter-8c66dbc4-r4fnp WhenEmptyOrUnderutilized 60s t3.medium,t3.small <u>1 <- 1 이상이면 정답 인정</u> ec2nodeclass.karpenter.k8s.aws/skm-app-nodeclass
3-6-A (명령어 입력)	<u>* 3-4, 3-5를 틀렸을 경우 채점을 진행하지 않습니다.</u> for b in \$(seq 1 10); do E=\$(for i in \$(seq 1 10); do printf '{"Id": "%d-%d", "MessageBody": "order"}', "\$b" "\$i"; done sed 's/,,\$//') aws sqs send-message-batch --queue-url "\$(aws sqs get-queue-url --queue-name skm-order-queue --region ap-northeast-2 --query QueueUrl --output text)" --entries "\$E" --region ap-northeast-2 > /dev/null; done
3-6-B (명령어 입력) <u>(최대 3분 대기)</u>	<u>* 부하 주입이 완료된 후, 아래 명령어를 실행합니다.</u> POD_PEAK=0; NODE_PEAK=0 for i in \$(seq 1 24); do P=\$(kubectl get deploy order-processor -n skillsmkt -o jsonpath='{status.readyReplicas}' 2>/dev/null); P=\${P:-0} N=\$(kubectl get nodes -l karpenter.sh/nodepool=skm-app-nodepool --no-headers 2>/dev/null wc -l tr -d ' ') ["\$P" -gt "\$POD_PEAK"] && POD_PEAK=\$P ["\$N" -gt "\$NODE_PEAK"] && NODE_PEAK=\$N sleep 5 done echo "Max Ready Pods \$POD_PEAK" echo "Max App Nodes \$NODE_PEAK"
3-6-B <u>부분 일치</u>	Max Ready Pods 5 <- 5 이상이면 정답 Max App Nodes 2 <- 2 이상이면 정답

4	3-7-A (명령어 입력)	* 3-4, 3-5, 3-6을 틀렸을 경우 채점을 진행하지 않습니다. aws sqs purge-queue --queue-url "\$(aws sqs get-queue-url --queue-name skm-order-queue --region ap-northeast-2 --query QueueUrl --output text)" --region ap-northeast-2 2>/dev/null
	3-7-B (명령어 입력) <u>(최대 3분 대기)</u>	* Queue Purge가 완료된 후, 아래 명령어를 실행합니다. for i in \$(seq 1 30); do P=\$(kubectl get deploy order-processor -n skillsmkt -o jsonpath='{.status.replicas}' 2>/dev/null); P=\${P:-0} N=\$(kubectl get nodes -l karpenter.sh/nodepool=skm-app-nodepool --no-headers 2>/dev/null wc -l tr -d ' ') if ["\$P" = "1"] && ["\$N" = "1"]; then RESULT=ok; break; fi sleep 5 done echo "Final Pods \$P" echo "Final Nodes \$N"
	3-7-B <u>정확히 일치</u>	Final Pods 1 Final Nodes 1
	4-0 사전준비	* 실행 중 에러가 발생할 경우 4모듈 채점을 진행하지 않습니다. source kubectl-connect o11y-cluster
	4-1-A (명령어 입력)	aws eks describe-cluster --name o11y-cluster --query 'cluster.[name, version, status]' --output text --region ap-northeast-1 aws eks describe-nodegroup --cluster-name o11y-cluster --nodegroup-name "\$(aws eks list-nodegroups --cluster-name o11y-cluster --region ap-northeast-1 --query 'nodegroups[0]' --output text)" --query 'nodegroup.[instanceTypes[0], scalingConfig.minSize, scalingConfig.desiredSize, scalingConfig.maxSize]' --output text --region ap-northeast-1 aws eks update-kubeconfig --name o11y-cluster --region ap-northeast-1 > /dev/null 2>&1 kubectl get nodes -o jsonpath='{range .items[*]}{.metadata.labels.topology\.kubernetes\.io/zone}{\ "n"}{end}' sort -u
	4-1-A <u>부분 일치</u>	o11y-cluster 1.35 ACTIVE t3.medium 2 2 2 ap-northeast-1a ap-northeast-1c * 파란색 부분의 경우 두 출력값이 다르다면 득점 처리합니다.
	4-2-A (명령어 입력)	ACCOUNT_ID=\$(aws sts get-caller-identity --query Account --output text) for n in o11y-app-alb o11y-grafana-alb; do; aws elbv2 describe-load-balancers --names \$n --query 'LoadBalancers[0].[State.Code, Type, Scheme]' --output text --region ap-northeast-1; done for n in o11y-app-tg o11y-grafana-tg; do; aws elbv2 describe-target-health --target-group-arn "\$(aws elbv2

		describe-target-groups --names \$n --query 'TargetGroups[0].TargetGroupArn' --output text --region ap-northeast-1" --query 'TargetHealthDescriptions[].TargetHealth.State' --output text --region ap-northeast-1; done
4-2-A <u>부분 일치</u>		active application internet-facing active application internet-facing <u>healthy healthy <- Healthy 이외의 값 출력되면 오답</u> <u>healthy <- Healthy 이외의 값 출력되면 오답</u>
4-3-A (명령어 입력)		kubectrl get deploy log-generator -n o11y -o custom-columns= 'NAME:.metadata.name,READY:.status.readyReplicas' kubectrl get ds o11y-otel -n monitoring -o custom-columns= 'NAME:.metadata.name,DESIRED:.status.desiredNumber Scheduled,READY:.status.numberReady' kubectrl get svc o11y-loki -n monitoring -o custom-columns= 'NAME:.metadata.name,TYPE:.spec.type,PORT:.spec.ports [0].port' kubectrl get deploy o11y-grafana -n monitoring -o custom-columns= 'NAME:.metadata.name,READY:.status.readyReplicas'
4-3-A <u>부분 일치</u>		NAME READY log-generator 2 NAME DESIRED READY o11y-otel 2 2 NAME TYPE PORT o11y-loki ClusterIP 3100 NAME READY o11y-grafana 1 <u>* 채점 시 빨간색 부분의 경우 1 이상이고, DESIRED와 READY 값이 일 치한 경우 득점 인정합니다.</u>
4-4-A (명령어 입력)		ALB=\$(aws elbv2 describe-load-balancers --names o11y-app-alb --query 'LoadBalancers[0].DNSName' --output text --region ap-northeast-1) curl -s "http://\$ALB/healthz"; echo curl -s "http://\$ALB/log?level= error& count= 3" head -1 jq -r '.level, .generated'
4-4-A <u>정확히 일치</u>		{"status": "ok"} error 3
4-5-A (명령어 입력) <u>(수동 채점)</u>		* mark4.sh의 하단에 해당 스크립트가 주석 처리되어 있으므로, 해당 스 크립트를 사용할 수 있음에 유의합니다. RESP=\$(curl -s "http://\$(aws elbv2 describe-load-balancers --names o11y-app-alb --query 'LoadBalancers[0].DNSName' --output text --region ap-northeast-1)/log?level= error& count= 3") kubectrl port-forward -n monitoring svc/o11y-loki 3100:3100 > /dev/null 2>&1 & PF=\$! sleep 2

		* 아래 명령어를 통해 로그가 정상적으로 조회되는지 확인합니다. 선수는 1분간 원하는 만큼 해당 명령어를 원하는 만큼 실행할 수 있습니다. <pre>curl -s -G http://localhost:3100/loki/api/v1/query_range --data-urlencode 'query={k8s_namespace_name="o11y"} json level="ERROR"' --data-urlencode "start=\$(date -d '3 minutes ago' +%s)000000000" --data-urlencode "end=\$(date +%s)000000000" --data-urlencode 'limit=20' jq -r '.data.result[0].values[0][1]'</pre> * 이후, 아래 명령어를 통해 포트포워딩 중인 프로세스를 종료합니다. <pre>kill \$PF 2>/dev/null</pre>
	4-5-A 부분 일치	<pre>{ "ts": "2026-05-31T12:26:32.805Z", "level": "ERROR", "msg": "log generated", "req_id": "75d9896e-db0a-4b98-b859-ea6df730b04f" } { "ts": "2026-05-31T12:26:32.805Z", "level": "ERROR", "msg": "log generated", "req_id": "9beba665-fb2b-4543-ade4-1cb87e793a44" }</pre> <u>* 출력되는 로그 중 3분 이내로 기록된 로그가 있다면 득점 인정.</u>
	4-6-A (명령어 입력) <u>(부분 점수)</u> <u>(수동 채점)</u>	* 4-2번 항목을 틀렸을 경우, 채점하지 않습니다. 선수의 Grafana ALB에 접속하여, skills<선수등번호> / GoodJob!Skills <선수등번호> ^ ^로 로그인한 뒤, Log Overview 대시보드를 엽니다. 1) 아래 사진과 같이 Panel 이름, 범례 등이 정상적으로 표시된다면 부분 점수 득점. (0.5점) 단, No Data로 표시되는 Panel이 하나라도 있거나, 범례가 {level="ERROR"} 등으로 표시된다면 오답 처리합니다. Panel은 아래 3개가 출력되어야 합니다. - Log Count Over Time - 막대그래프 형식의 패널 - Log Level Distribution - 원그래프 형식의 패널 - Recent Logs - 집계된 로그 출력 2) 아래 Recent Logs에 4-5에서 전송한 로그가 존재하는지 확인합니다. 존재한다면 부분 점수 득점 인정합니다 (0.5점)   > Data Sources로 접속하여 Loki Source를 선택합니다. 만약에 없다면, 선수가 지정한 source를 선택합니다. (단, 선택을 바꿀 수 없습니다.) 페이지 최하단으로 이동하여 Save& Test를 클릭합니다.

		아래와 같이 성공으로 표시되면 부분 점수 득점 인정합니다. (0.5점) 
--	--	--

3) 채점 내용